

Chapter 1

Introduction to Cryptography

Stefan Dziembowski

www.crypto.edu.pl/Dziembowski

University of Warsaw



Basic information

- **Exam:** written – theory and exercises
- **Website:** github.com/sdziembowski/cryptography-I-2026
- **Main handbook:** Jonathan Katz and Yehuda Lindell [Introduction to Modern Cryptography](#)
- **Other books:**
 - Mike Rosulek [The Joy of Cryptography](#)
 - Dan Boneh and Victor Shoup [A Graduate Course in Applied Cryptography](#)

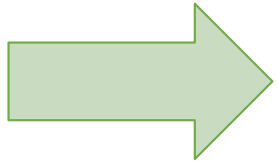
Basic information

Lecturer: Stefan Dziembowski

TA: Mahesh Rajasree



Plan



1. Introduction
 1. What is cryptography?
 2. Plan of the course
2. Historical ciphers
3. Information-theoretic security
4. Computational security

Historical cryptography

cryptography $\approx$ encryption

main applications: **military and diplomacy**



Modern cryptography

cryptography = much more
than encryption!



indistinguishability obfuscation

mental poker

signature schemes

electronic auctions

key agreement

e-cash

electronic voting

zero-knowledge

public-key
cryptography

multiparty-computations

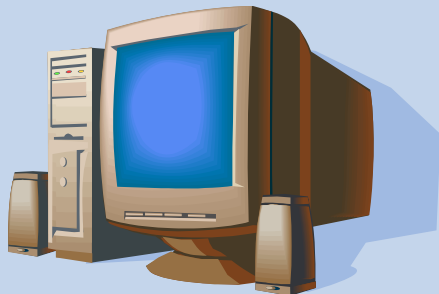
seventies

now

What happened in the seventies?

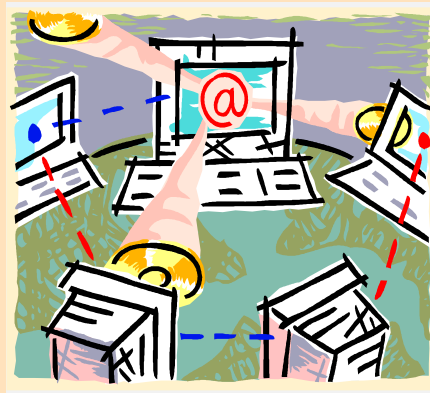
Technology

affordable
hardware



Demand

companies and
individuals start
to do business
electronically



Theory

**the
computational
complexity
theory is born**

this allows
researchers to
reason about
security in a
formal way.

Cryptography



In the past:

the **art** of encrypting messages (mostly for the military applications).

Now:



the **science** of securing digital communication and transactions (encryption, authentication, digital signatures, e-cash, auctions, etc..)

Do citizens have a right to analyse and use strong cryptography?

Main **opponents**:

- **Governments** and their **agencies** (most notably: the US **National Security Agency**).
 - **control proliferation to other countries** (mainly in the past)
 - tracing **criminal** and **terrorist** activities
- Some corporations:
 - **copyright protection** (see, e.g., [Digital Millennium Copyright Act](#))

Main **proponents**:

- **academics**
- **civil liberties organizations** (e.g., [Electronic Frontier Foundation](#))

“Crypto wars”

Good crypto implemented correctly is impossible to break even for governmental agencies.

Attempts to circumvent it:

- **criminalizing breaking cryptographic algorithms**
- **export control:**
 - mainly in the past,
 - possible to **bypass** using the US First Amendment (freedom of speech), see, e.g., the history of [PGP](#)
- **weak crypto**
- **artificially short keys**
- **official backdoors**
 - key escrow (see, e.g., the [Clipper chip](#))
 - very recently: [the EU's chat control directive](#) (see also: <https://csa-scientist-open-letter.org/Sep2025>)
- **unofficial backdoors**
 - [DUAL EC DRGB](#) – we will look at it later
- supply chain attacks, side channel attacks, malware attacks,...

} — we will see examples

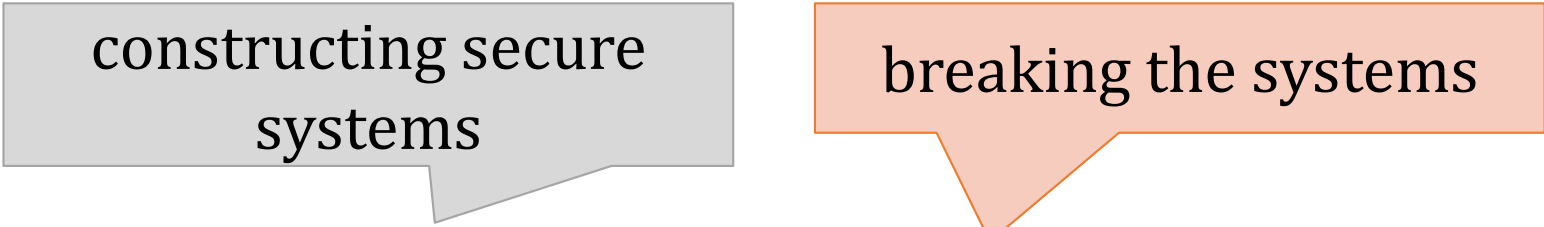
Arguments for crypto control

- In democratic countries, the **governmental agencies are the “good guys”** and are controlled.
- Digital crime is a considerable problem (e.g., **child abuse, terrorism**).
- **Copyright owners need protection.**

Arguments against crypto control

- Privacy as a **fundamental citizen's right**:
(similar, e.g., to a right to a fair trial)
 - even democratic countries have a long history of privacy abuse
 - security agencies have incentives to “take shortcuts”
- Impossible to **limit to one state**:
 - technology is borderless
 - Even allies spy on each other (especially: industrial espionage)
- Introduces **additional attack vectors**

Terminology



constructing secure
systems

breaking the systems

Cryptology = cryptography + cryptanalysis

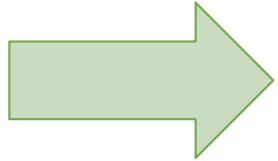
This convention is **slightly artificial** and often ignored.

Common usage:

“cryptanalysis of X” = “breaking X”

Common abbreviation: **“crypto”**

Plan



1. Introduction
 1. What is cryptography?
 2. Plan of the course
2. Historical ciphers
3. Information-theoretic security
4. Computational security

Three components of the course

1. practical aspects

2. mathematical foundations

3. new horizons

Practical aspects

- **symmetric encryption**: block ciphers and stream ciphers
- **hash functions**
- **message authentication**
- **public-key infrastructure**
- **elements of number theory**
- **asymmetric encryption**
- **signature schemes**

Mathematical foundations

- What makes us believe that the **protocols are secure**?
- Can we formally **define** “security”?
- Can security be **proven**?
- Do there exist “**unbreakable**” ciphers?

New horizons

Advanced cryptographic protocols, such as:

- **zero-knowledge**
- **multiparty computations**
- **private information retrieval**



This course is **not** about

- **practical data security** (firewalls, intrusion-detection, VPNs, etc.),
- **history** of cryptography,
- **number theory** and **algebra**
(we will use them **only as tools**)
- **complexity theory**
- **cryptocurrencies and blockchain**

Cryptography – general picture

plan of the course:

	encryption	authentication
private key	1 private key encryption	3 private key authentication
public key	4 public key encryption	5 signatures

2 hash functions

Cryptography II
(summer semester)

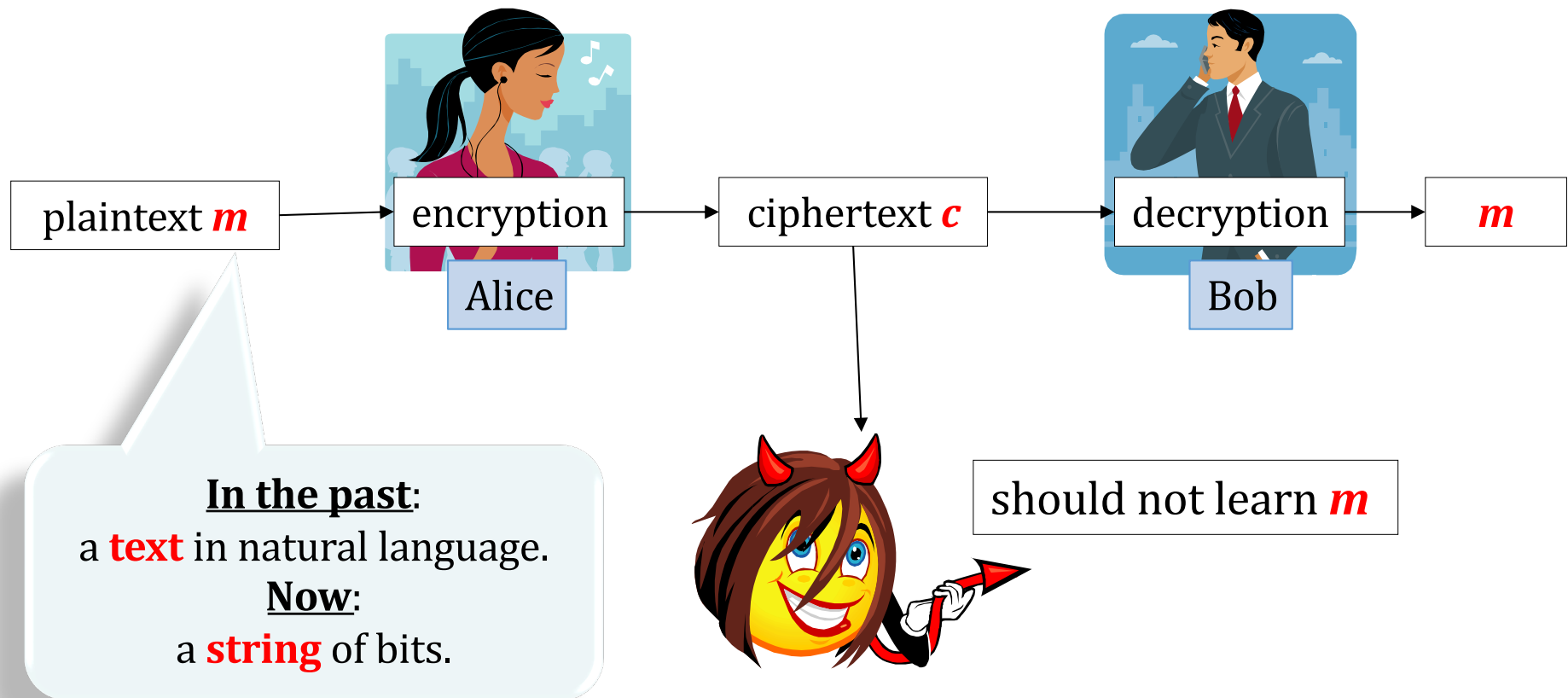
6 advanced cryptographic protocols

Preliminary plan of the lectures

1. Introduction to Cryptography
2. Symmetric Encryption
3. Hash Functions and Message Authentication
4. Introduction to Public-Key Cryptography
5. A Brush-up on Number Theory and Algebra
6. Public-Key Encryption
7. Signature Schemes
8. Introduction to the Advanced Protocols

Encryption schemes (a very general picture)

Encryption scheme (cipher) = encryption & decryption



Art vs. science

In the past:

lack of precise definitions, ad-hoc design, usually insecure.

Nowadays:

formal definitions, systematic design, very secure constructions.

Provable security

We want to construct schemes that are **provably secure**.

But...

- **why** do we want to do it?
- **how** to define it?
- and is it **possible** to achieve it?

Provable security – the motivation

In many areas of computer science formal proofs are **not essential**.

For example, instead of proving that an algorithm is efficient, we can just simulate it on a “*typical*” input”.

In **cryptography** it's **not true**, because

there cannot exist an experimental proof that a scheme is secure.

Why?

Because a notion of a

“*typical*” adversary”

does not make sense.

Security definitions are useful also because they allow us to construct schemes in a modular way...

Kerckhoffs' principle



Auguste Kerckhoffs (1883):

The enemy knows the system

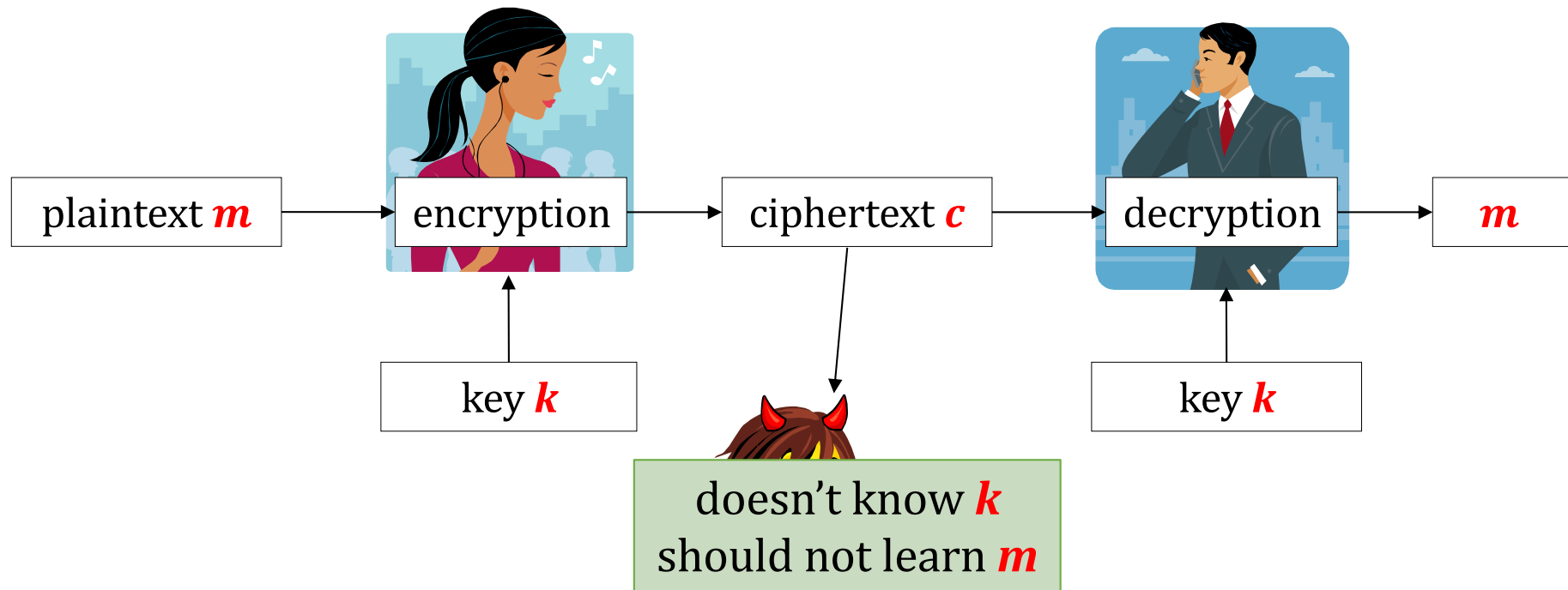
The cipher should remain secure
even if **the adversary knows the
specification of the cipher.**

The only thing that is **secret** is a

short key ***k***

that is **usually chosen uniformly at random**

A more refined picture



- Of course, Bob can use the same method to send messages to Alice. (that's why it's called the **symmetric setting**)
- For a moment assume that we care only about **sending one message**

Assume k is uniformly random

Kerckhoffs' principle – the motivation

1. In commercial products it is unrealistic to assume that the design details remain secret (**reverse-engineering!**)
2. Short keys are easier to **protect, generate** and **replaced**.
3. The design details can be discussed and **analyzed in public**.

Not respecting this principle
=
“security by obscurity”.

A mathematical view

$\mathcal{K}$ – **key** space

$\mathcal{M}$ – **plaintext** space

$\mathcal{C}$ – **ciphertext** space

An **encryption scheme** is a pair **(Enc, Dec)**, where

- **Enc** : $\mathcal{K} \times \mathcal{M} \rightarrow \mathcal{C}$ is an **encryption** algorithm,
- **Dec** : $\mathcal{K} \times \mathcal{C} \rightarrow \mathcal{M}$ is an **decryption** algorithm.

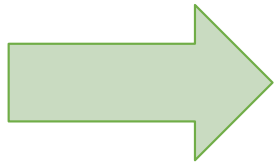
We will sometimes write **Enc_k(m)** and **Dec_k(c)** instead of **Enc(k,m)** and **Dec(k,c)**.

Correctness

for every k we should have **Dec_k(Enc_k(m)) = m**.

Plan

1. Introduction



2. Historical ciphers

3. Information-theoretic security

4. Computational security

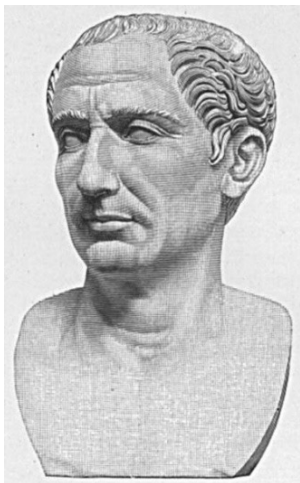
Shift cipher

$\mathcal{M}$ = words over alphabet $\{A, \dots, Z\} \approx \{0, \dots, 25\}$

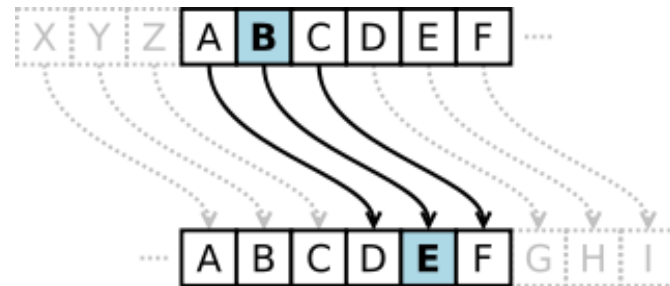
$\mathcal{K} = \{0, \dots, 25\}$

$\text{Enc}_k(m_0, \dots, m_n) = (m_0 + k \bmod 26, \dots, m_n + k \bmod 26)$

$\text{Dec}_k(c_0, \dots, c_n) = (c_0 - k \bmod 26, \dots, c_n - k \bmod 26)$



Caesar: $k = 3$



Security of the shift cipher

How to break the shift cipher?

Check all possible keys!

Let c be a ciphertext.

For every $k \in \{0, \dots, 25\}$ check if $\text{Dec}_k(c)$ “makes sense”.

Most probably only one such k exists.

Thus $\text{Dec}_k(c)$ is the message.

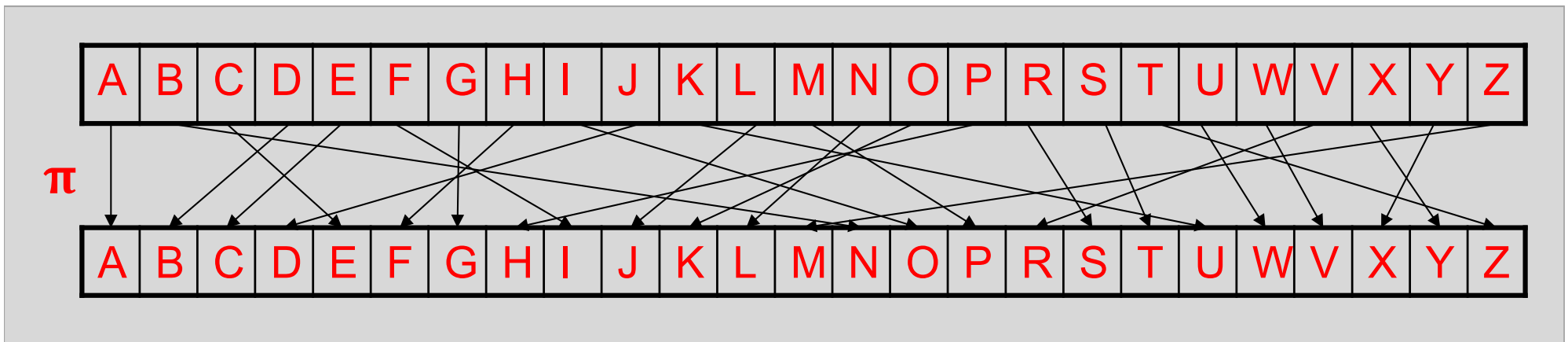
This is called a **brute force attack**.

Moral: the key space needs to be large!

Substitution cipher

$\mathcal{M}$ = words over alphabet $\{A, \dots, Z\} \approx \{0, \dots, 25\}$

$\mathcal{K}$ = a set of permutations of $\{0, \dots, 25\}$



$$\text{Enc}_{\pi}(m_0, \dots, m_n) = (\pi(m_0), \dots, \pi(m_n))$$

$$\text{Dec}_{\pi}(c_0, \dots, c_n) = (\pi^{-1}(c_0), \dots, \pi^{-1}(c_n))$$

How to break the substitution cipher?

Use **statistical patterns** of the language.

For example: the **frequency tables**.

Texts of **50** characters can usually be broken this way.

Letter	Frequency
E	0.127
T	0.097
I	0.075
A	0.073
O	0.068
N	0.067
S	0.067
R	0.064
H	0.049
C	0.045
L	0.040
D	0.031
P	0.030
Y	0.027
U	0.024
M	0.024
F	0.021
B	0.017
G	0.016
W	0.013
V	0.008
K	0.008
X	0.005
Q	0.002
Z	0.001
J	0.001

Figure 7 - Frequency Table

Other famous historical ciphers

Vigenère cipher:



Blaise de Vigenère
(1523 - 1596)

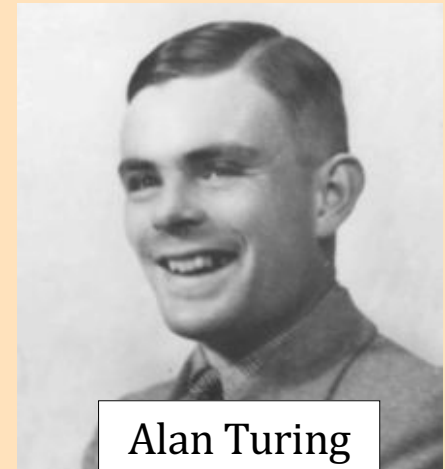


Leon Battista Alberti
(1404 - 1472)

Enigma

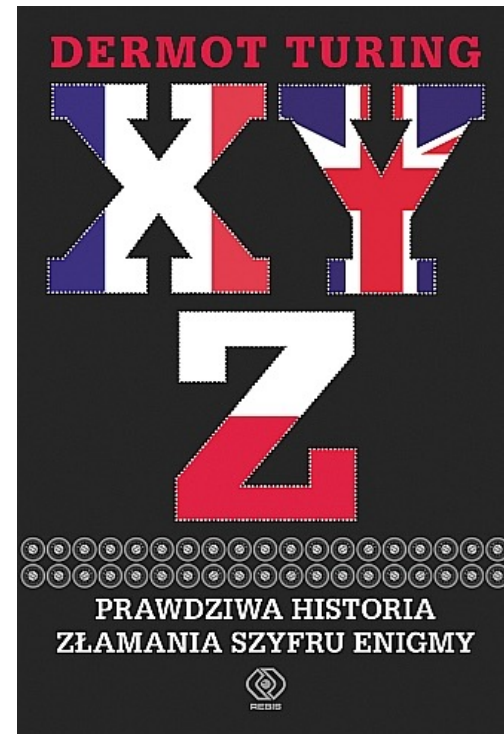
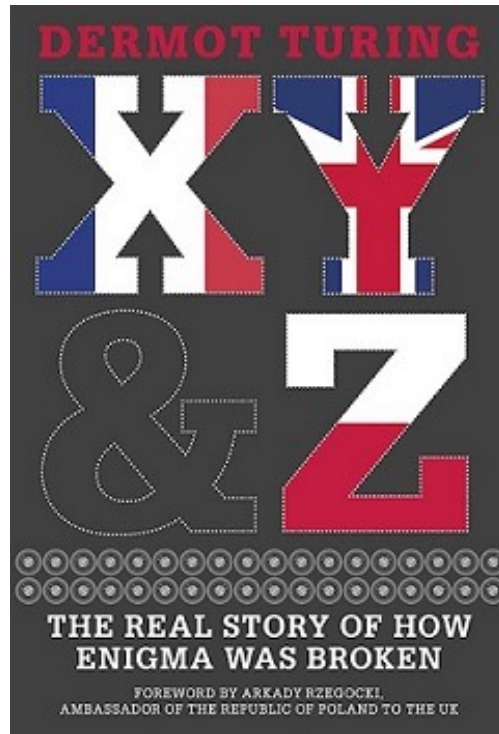


Marian Rejewski
(1905 - 1980)



Alan Turing
(1912-1954)

A highly recommended historical book on breaking Enigma



Dermot Turing: **X, Y & Z: The Real Story of How Enigma Was Broken** (The History Press, 2018)

In the past ciphers were designed in an ad-hoc manner

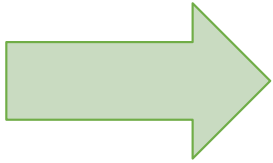
In contemporary cryptography the ciphers are designed in a **systematic way**.

Main goals:

1. define security
2. construct schemes that are “provably secure”

Plan

1. Introduction
2. Historical ciphers
3. Information-theoretic security
4. Computational security



Defining “security of an encryption scheme” is not trivial.

consider the following experiment

(m – a message)

1. the key K is chosen uniformly at random
2. $C := \text{Enc}_K(m)$ is given to the adversary

how to define
security

?

Idea 1

(m – a message)

1. the key K is chosen uniformly at random
2. $C := \text{Enc}_K(m)$ is given to the adversary

An idea

“The adversary should not be able to compute K .”

A problem

the encryption scheme that “doesn’t encrypt”:

$$\text{Enc}_K(m) = m$$

satisfies this definition!



Idea 2

(m – a message)

1. the key K is chosen uniformly at random
2. $C := \text{Enc}_K(m)$ is given to the adversary

An idea

“The adversary should not be able to compute m .”

A problem

What if the adversary can compute, e.g., the first half of m ?



Idea 3

(m – a message)

1. the key K is chosen uniformly at random
2. $C := \text{Enc}_K(m)$ is given to the adversary

An idea

“The adversary should not learn any information about m .”

A problem

But he may already have some a priori information about m !

For example he may know that m is a sentence in English...



Idea 4

(m – a message)

1. the key K is chosen uniformly at random
2. $C := \text{Enc}_K(m)$ is given to the adversary

An idea

“The adversary should not learn any additional information about m .”

This makes much more sense.

But how to formalize it?

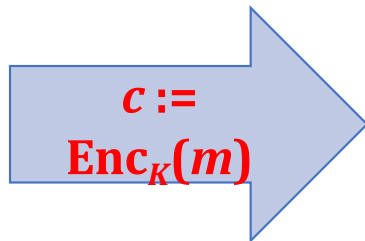
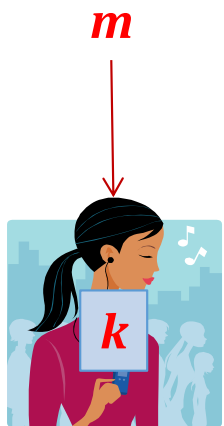


Example



Eve knows that

$m := \begin{cases} \text{"I love you"} & \text{with prob. } \mathbf{0.1} \\ \text{"I don't love you"} & \text{with prob. } \mathbf{0.7} \\ \text{"I hate you"} & \text{with prob. } \mathbf{0.2} \end{cases}$



Eve **still** knows that

$m := \begin{cases} \text{"I love you"} & \text{with prob. } \mathbf{0.1} \\ \text{"I don't love you"} & \text{with prob. } \mathbf{0.7} \\ \text{"I hate you"} & \text{with prob. } \mathbf{0.2} \end{cases}$

How to formalize the “Idea 4”?

“The adversary should not learn any additional information about m .”

also called: **information-theoretically** secret

An encryption scheme is **perfectly secret** if

for every random variable M

and every $m \in \mathcal{M}$ and $c \in \mathcal{C}$

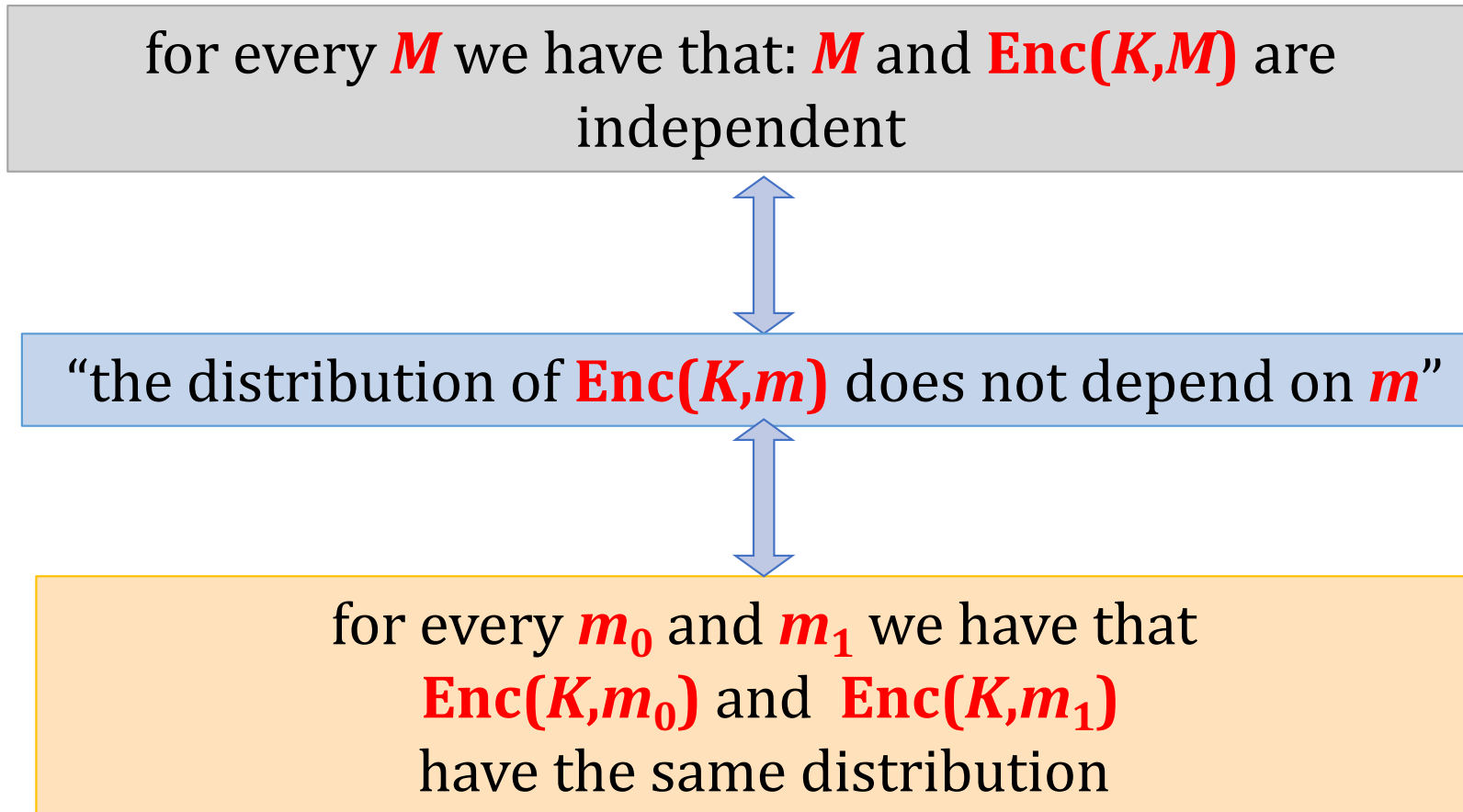
such that
 $P(C = c) > 0$

$$P(M = m) = P(M = m \mid (\text{Enc}(K, M)) = c)$$



equivalently: M and $\text{Enc}(K, M)$ are independent

Equivalently:



A perfectly-secret scheme: one-time pad

t – a parameter
 $\mathcal{K} = \mathcal{M} = \{0,1\}^t$

component-wise **xor**

Vernam's cipher:

$$\text{Enc}_k(m) = k \text{ xor } m$$

$$\text{Dec}_k(c) = k \text{ xor } c$$



Gilbert
Vernam
(1890 –1960)

Correctness is trivial:

$$\text{Dec}_k(\text{Enc}_k(m)) = k \text{ xor } (k \text{ xor } m) \\ m$$

Perfect secrecy of the one-time pad

Perfect secrecy of the one time pad is also trivial.

This is because for every m
the distribution of $\text{Enc}(K, m)$ is uniform
(and hence does not depend on m).

for every c :

$$P(\text{Enc}(K, m) = c) = P(K = m \text{ xor } c) = 2^{-t}$$

Observation

One time pad can be **generalized** as follows.

Let $(G, +)$ be a group. Let $\mathcal{K} = \mathcal{M} = \mathcal{C} = G$.

The following is a perfectly secret encryption scheme:

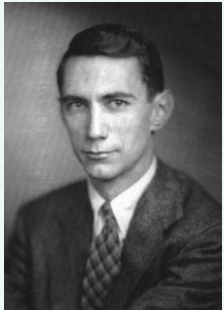
- $\text{Enc}(k, m) = m + k$
- $\text{Dec}(k, m) = m - k$

Why the one-time pad is not practical?

1. The key has to be as long as the message.
2. The key cannot be reused

This is because:

$$\begin{aligned}\text{Enc}_k(m_0) \text{ xor } \text{Enc}_k(m_1) &= (k \text{ xor } m_0) \text{ xor } (k \text{ xor } m_1) \\ &= m_0 \text{ xor } m_1\end{aligned}$$



Theorem (Shannon 1949)

(“One time-pad is optimal in the class of perfectly secret schemes”)

In every perfectly secret encryption scheme

$$\text{Enc} : \mathcal{K} \times \mathcal{M} \rightarrow \mathcal{C}, \text{Dec} : \mathcal{K} \times \mathcal{C} \rightarrow \mathcal{M}$$

we have $|\mathcal{K}| \geq |\mathcal{M}|$.

Proof

Perfect secrecy implies that the distribution of $\text{Enc}(K, m)$ does not depend on m . Hence for every m_0 and m_1 we have

$$\{\text{Enc}(k, m_0)\}_{k \in \mathcal{K}} = \{\text{Enc}(k, m_1)\}_{k \in \mathcal{K}}$$

denote this set with $\mathcal{C}'$

Observation: $|\mathcal{K}| \geq |\mathcal{C}'|$.

Fact: we always have that $|\mathcal{C}'| \geq |\mathcal{M}|$.

This is because for every k we have that

$\text{Enc}_k : \mathcal{M} \rightarrow \mathcal{C}'$ is an injection

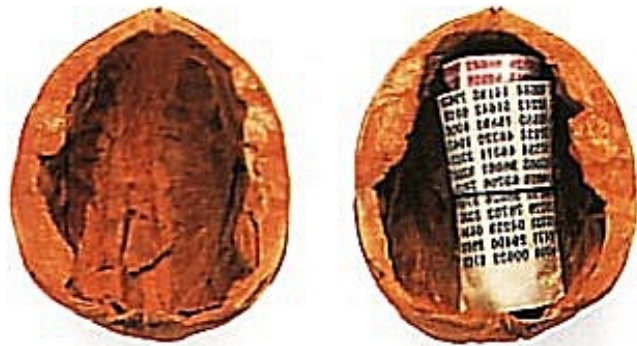
(otherwise we wouldn't be able to decrypt).

$$|\mathcal{K}| \geq |\mathcal{M}|$$

Practicality?

Generally, the **one-time pad** is **not very practical**, since:

- the key has to be as long as the **total** length of the encrypted messages,
- it is hard to generate truly random strings.



a **KGB** one-time pad hidden
in a walnut shell

However, it is sometimes used (e.g. in the **military applications**), because of the following advantages:

- **perfect secrecy**,
- short messages can be encrypted using **pencil and paper** .

In the 1960s the Americans and the Soviets established a hotline that was encrypted using the one-time pad. (**additional advantage**: they didn't need to share their secret encryption methods)

Venona project (1946 – 1980)



Ethel and Julius Rosenberg

American **National Security Agency** decrypted **Soviet** messages that were transmitted in the 1940s.

That was possible because the Soviets reused the keys in the one-time pad scheme.

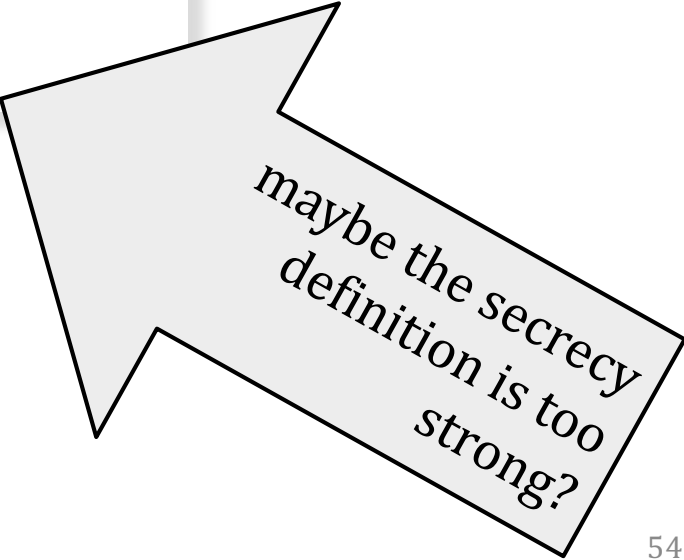
Outlook

We constructed a perfectly secret encryption scheme

Our scheme has certain drawbacks ($|\mathcal{K}| \geq |\mathcal{M}|$).

But by Shannon's theorem this is unavoidable.

Can we go home and relax?



maybe the secrecy
definition is too
strong?

What to do?

Idea

use a model where the **power** of the adversary is limited.

How?

Classical (computationally-secure) cryptography:

bound his computational power.

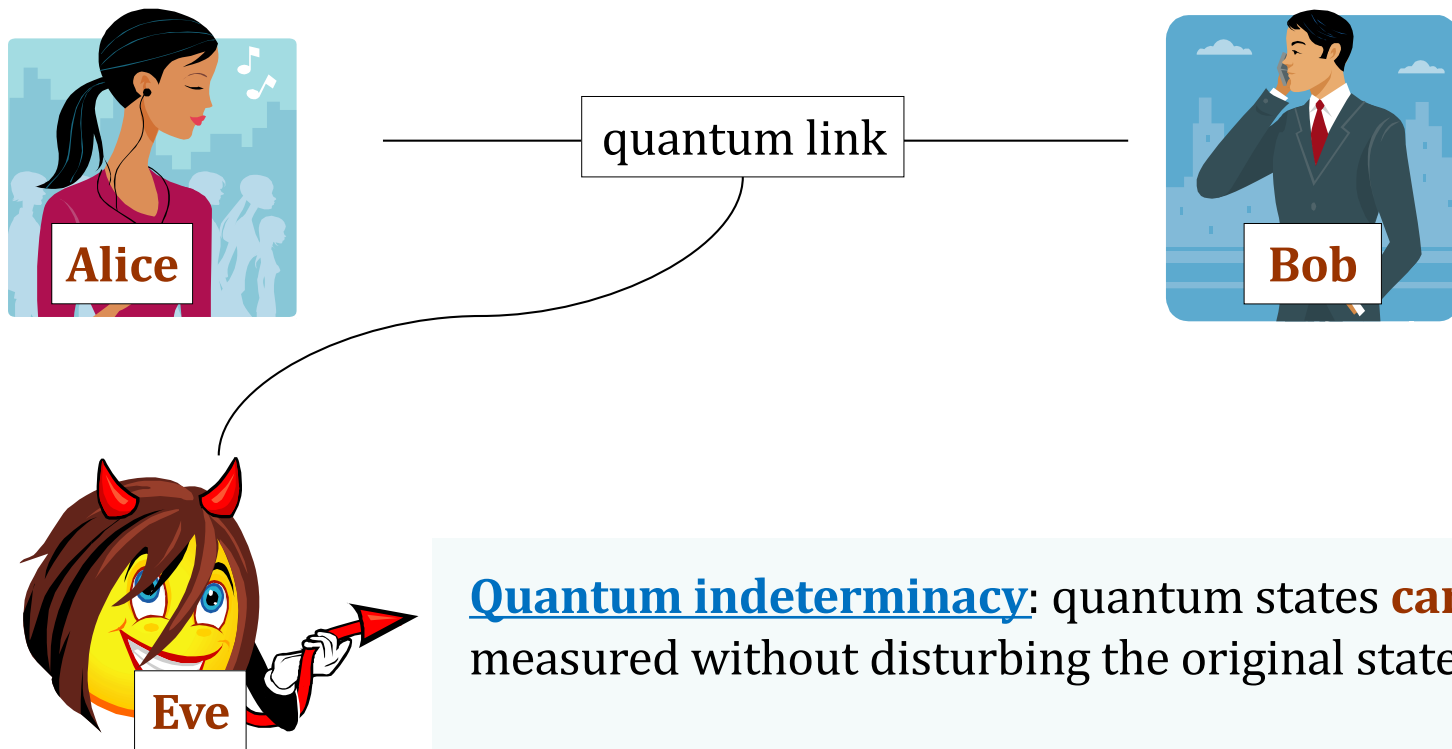
Alternative options:

quantum cryptography, bounded-storage model,...

(not too practical)

Quantum cryptography

Stephen Wiesner (1970s), Charles H. Bennett and Gilles Brassard (1984),
Artur Ekert (1991)



Quantum indeterminacy: quantum states **cannot** be measured without disturbing the original state.

Hence **Eve** cannot read the bits in an unnoticeable way.

Quantum cryptography

Advantage: **security is based on the laws of quantum physics**

Disadvantages:

- **needs a dedicated equipment**
- may be target to attacks that are not captured by the model (“**quantum hacking**”)
- a problem constructing **untrusted quantum relays**
- **wireless transmission** creates additional problems

Practicality?

Currently: successful transmissions for distances of length hundreds of kilometres.

Statements by security agencies

Consensus: **quantum cryptography is currently pure theory without practical applications:**



US National Security Agency:

www.nsa.gov/Cybersecurity/Quantum-Key-Distribution-QKD-and-Quantum-Cryptography-QC/

For a rebuttal by physicists see: Renato Renner and Ramona Wolf
The debate over QKD: A rebuttal to the NSA's objections
<https://arxiv.org/pdf/2307.15116.pdf>



Federal Office
for Information Security



General Intelligence and
Security Service
Ministry of the Interior and
Kingdom Relations



SWEDISH ARMED FORCES

French, German, UK, and Swedish security agencies

https://cyber.gouv.fr/sites/default/files/document/Quantum_Key_Distribution_Position_Paper.pdf

Terms not to confuse

1. **Quantum computing** – using quantum mechanics to **quantum computers**

do not exist (yet) – see “quantum supremacy”

preferred
by the
physicists

2. **Quantum cryptography** – using quantum mechanics to construct cryptographic schemes that are secure against any (quantum or not) computers

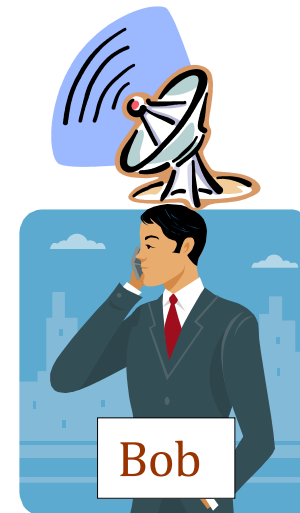
preferred
by the
security
experts

3. **Post-quantum cryptography** – constructing classical (i.e., computationally secure) cryptographic schemes that are **secure against quantum computers**.

A satellite scenario

A third party (a satellite) is broadcasting random bits.

```
000110100111010010011010111001110111
111010011101010101010010010100111100
001001111111100010101001000101010010
001010010100101011010101001010010101
```

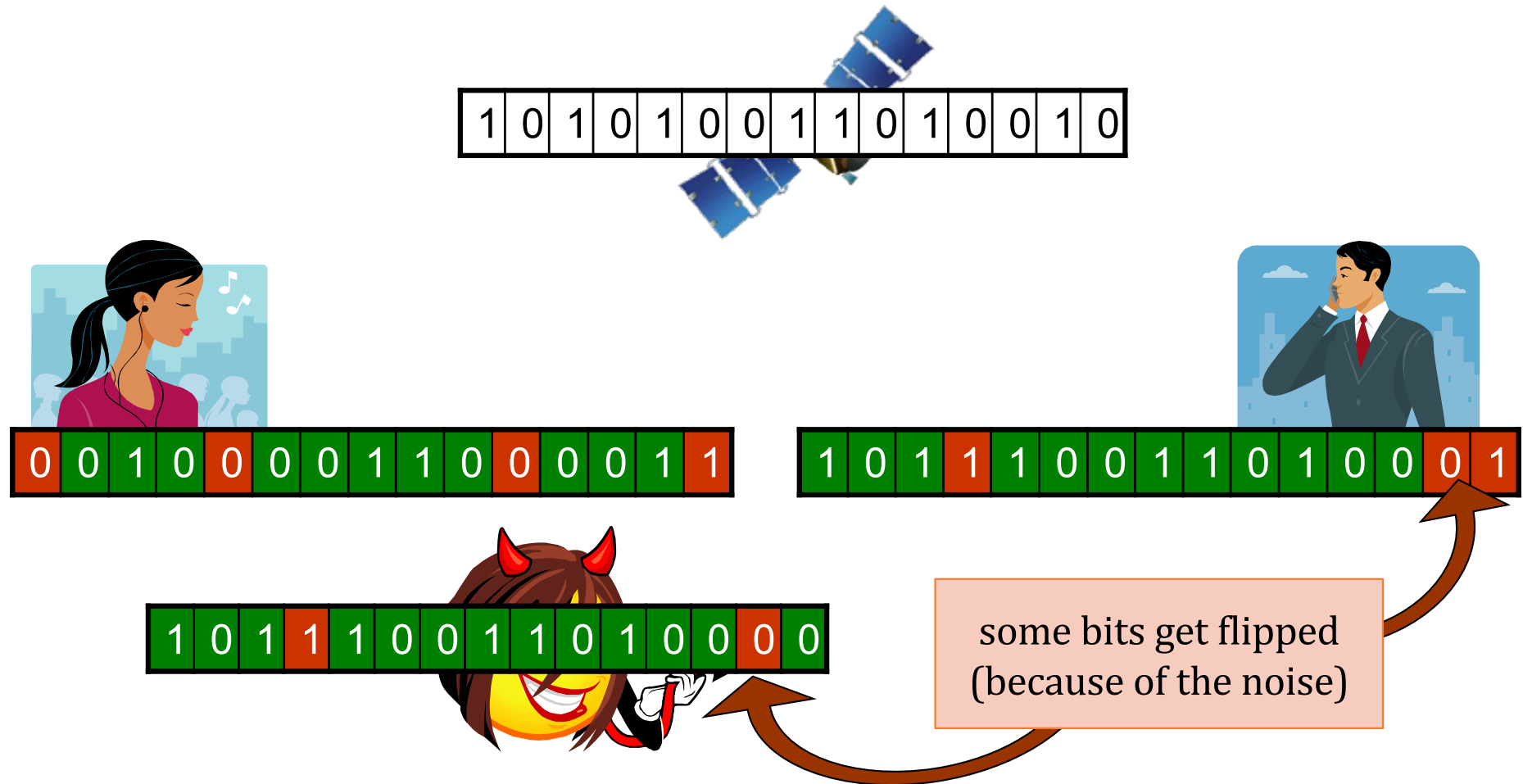


Does it help?

No...

(**Shannon's theorem** of course also holds in this case.)

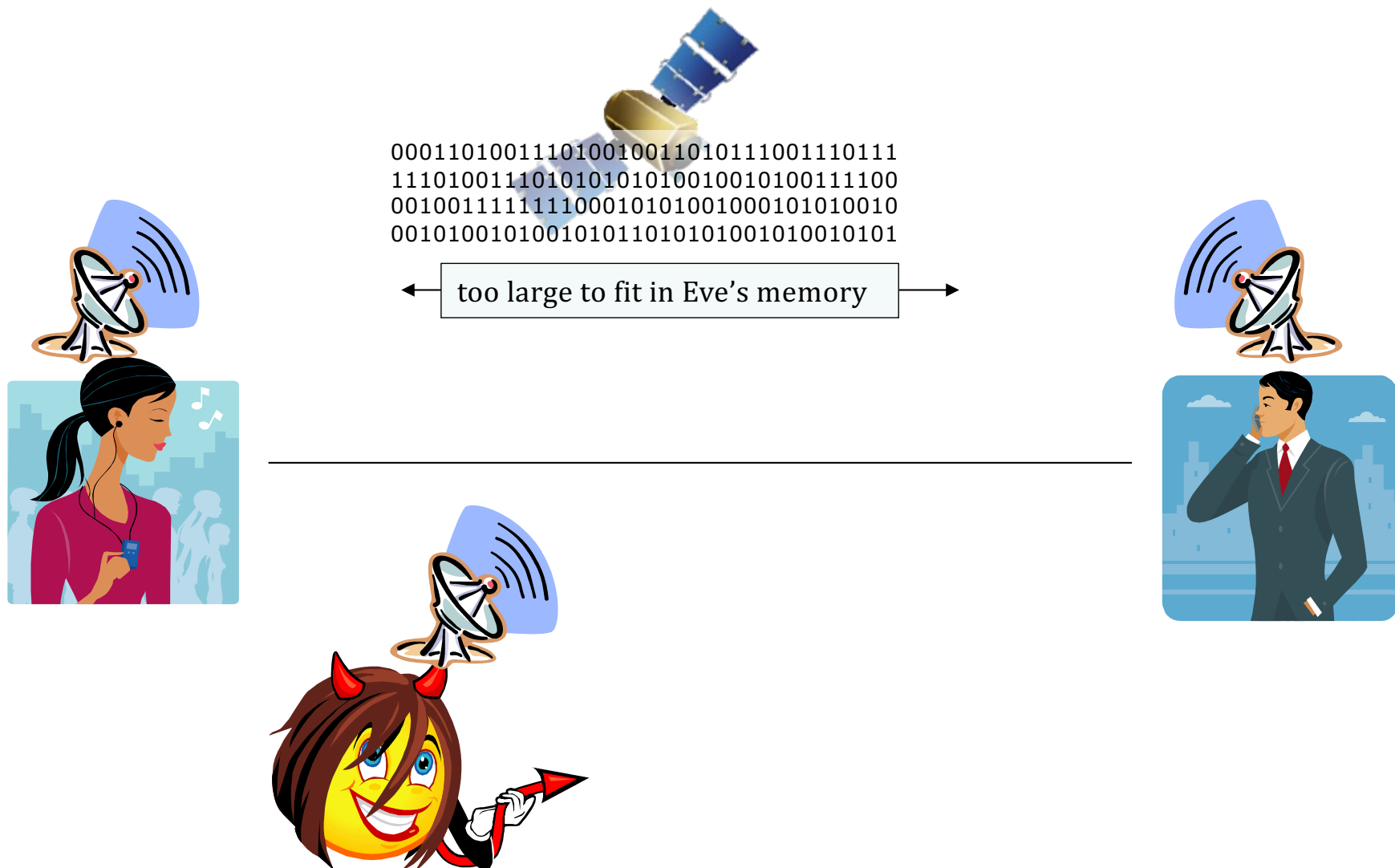
Ueli Maurer (1993): noisy channel.



Assumption: the data that the adversary receives is noisy.
(The data that Alice and Bob receive may be even more noisy.)

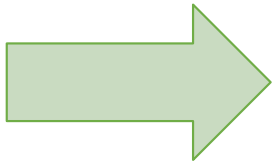
Bounded-Storage Model

Another idea: bound the size of adversary's memory



Plan

1. Introduction
2. Historical ciphers
3. Information-theoretic security
4. **Computational security**



How to reason about the bounded computing power?

perfect secrecy:
 M and $\text{Enc}_K(M)$
are independent

It is enough to require that

M and $\text{Enc}_K(M)$
are independent
*“from the point of view of a computationally-limited
adversary”.*

How can this be formalized?

We will use the **complexity theory!**

Real cryptography starts here:



Eve is computationally-bounded

We will construct schemes that in **principle can be broken** if the adversary has a huge computing power.

For example, the adversary will be able to break the scheme by enumerating all possible secret keys.

(this is called a “**brute force attack**”)

Computationally-bounded adversary



Eve is computationally-bounded

But what does it mean?

Ideas:

1. "She has can use at most **1000 Intel Core i9-13900K** for at most **100** years..."
2. "She can buy equipment worth **1 million euro** and use it for **30** years..".

it's hard to reason
formally about it

A better idea

"The adversary has access to a **Turing Machine** that can make at most 10^{30} steps."

More generally, we could have definitions of a type:

"a system **X is (t, ϵ) -secure** if every **Turing Machine**
that operates in time **t**
can break it with probability at most **ϵ** ."

This would be quite precise, **but...**

We would need to specify exactly what we mean by a "**Turing Machine**":

- **how many tapes does it have?**
- how does it access these tapes (maybe a "**random access memory**" is a more realistic model..)
- ...

*Moreover, this approach often leads to **ugly formulas**...*

What to do?

Idea

:

t steps of a Turing Machine $\rightarrow$ “**efficient computation**”

$\epsilon \rightarrow$ a value “**very close to zero**”.

How to formalize it?

Use the **asymptotics**!

Efficiently computable?

“efficiently computable”

=

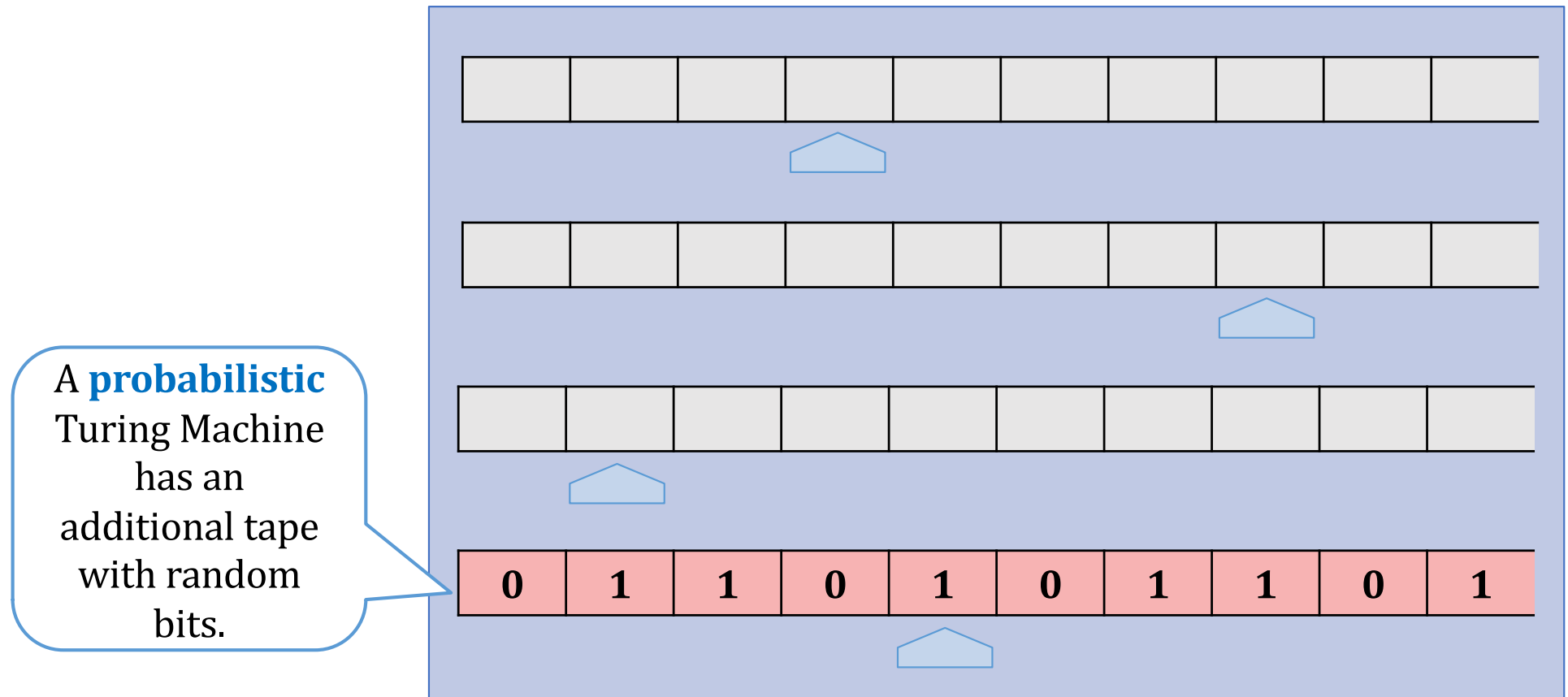
“polynomial-time computable
on a **Probabilistic Turing
Machine**”

that is: running in time
 $O(n^c)$ (for some **c**)

Here we assume that the **poly-time Turing Machines**
are the right model for the real-life computation.

Probabilistic Turing Machines

A standard Turing Machine has some number of tapes:



Some notation

If M is a Turing Machine then

$$M(X)$$

is a **random variable** denoting the **output** of M
assuming that
the contents of the random tape was chosen
uniformly at random.

More notation

$$Y \leftarrow M(X)$$

means that the variable Y takes the value that M outputs on input X (assuming the random input is chosen uniformly).

If $\mathcal{A}$ is a set then

$$Y \leftarrow \mathcal{A}$$

means that Y is chosen uniformly at random from the set $\mathcal{A}$.

Very small?

“very small”

=

“negligible”

=

approaches 0 faster than the inverse of any polynomial

Formally

A function $\mu : \mathbb{N} \rightarrow \mathbb{R}$ is negligible if for every positive integer c there exists an integer N such that for all $x > N$

$$|\mu(x)| \leq \frac{1}{x^c}$$

Negligible or not?

$$f(n) := \frac{1}{n^2} \quad \text{no}$$

$$f(n) := 2^{-n} \quad \text{yes}$$

$$f(n) := 2^{-\sqrt{n}} \quad \text{yes}$$

$$f(n) := n^{-\log n} \quad \text{yes}$$

$$f(n) := \frac{1}{n^{1000}} \quad \text{no}$$

Nice properties of these notions

A sum of two polynomials is a polynomial:

$$\text{poly} + \text{poly} = \text{poly}$$

A product of two polynomials is a polynomial:

$$\text{poly} * \text{poly} = \text{poly}$$

A sum of two negligible functions is a negligible function:

$$\text{negl} + \text{negl} = \text{negl}$$

Moreover

A negligible function multiplied by a polynomial is negligible

$$\text{negl} * \text{poly} = \text{negl}$$

Security parameter

Typically, we will say that a scheme X is secure if



P (M breaks the scheme X) is negligible

polynomial-time
Turing Machine M

The terms “negligible” and “polynomial” make sense only if X and the adversary take an additional input 1^n called a **security parameter**.

In other words: we consider an infinite sequence $X(1^1), X(1^2), \dots$ of schemes.

A common convention in symmetric encryption

$$\mathcal{K} = \{0, 1\}^n$$

security parameter

$$\mathcal{M} = \mathcal{C} = \{0, 1\}^*$$

Example

security parameter 1^n – n is the length of the secret key k

in other words: k is always a random element of $\{0,1\}^n$

The adversary can always **guess** k with probability 2^{-n} .

This probability **is negligible**.

He can also **enumerate all possible keys** k in time 2^n .
(the “brute force” attack)

This time **is exponential**.

Is this the right approach?

Advantages



1. All types of **Turing Machines** are “equivalent” up to a “**polynomial reduction**”.
Therefore we do need to specify the details of the model.
2. The formulas get much simpler.

Disadvantage



Asymptotic results don't tell us anything about security of the **concrete systems**.



However

Usually one can prove **formally** an asymptotic result and then argue **informally** that “the constants are reasonable”
(and can be calculated if one really wants).

©2025 by Stefan Dziembowski. Permission to make digital or hard copies of part or all of this material is currently granted without fee *provided that copies are made only for personal or classroom use, are not distributed for profit or commercial advantage, and that new copies bear this notice and the full citation.*